

Lab 01 — IAM & Account Security Design

Practise IAM users, groups, roles, least-privilege policies and explicit DENY against a local IAM and S3 emulator — no AWS account needed.

Lab environment: Docker Desktop + LocalStack — runs locally, no AWS account needed

Estimated time: 35–40 minutes

SAA-C03 domain: Design Secure Architectures

Overview

Local component	AWS equivalent
<code>dev1</code> , <code>operator1</code> , <code>auditor1</code>	IAM users
<code>Developers</code> , <code>Operators</code> , <code>ReadOnly</code>	IAM groups
<code>DeveloperS3ReadOnly</code> (with <code>ExplicitDenyDelete</code>)	Customer-managed IAM policy
<code>AppRole</code> (trusted by EC2)	IAM role for an application
<code>lab1-secure-data</code>	Amazon S3 bucket

These resources are created automatically by `scripts/init-aws.sh` when LocalStack starts. MFA, IAM Access Analyzer, CloudTrail and Organizations/SCPs are outside the scope of this local lab.

Step 1 — Get the lab source and start the lab

The repo lives in `~/Downloads/aws-saa`. The paths are absolute, so these commands work from any folder. `git clone` is needed only once: if the repo is already there it prints `already exists`, and the next line still takes you into this lab.

```
cd ~/Downloads
git clone https://github.com/mohanpothula/aws-saa.git
cd ~/Downloads/aws-saa/labs/lab-01-iam-account-security
docker compose up -d
# wait for the init script to finish creating the IAM resources
until docker compose logs localstack 2>&1 | grep -q "created successfully"; do sleep 2; done;
echo READY
docker exec -it aws-saa-lab1 bash
```

LocalStack reports *healthy* several seconds before its init script has finished creating the users and groups. Wait for `READY` before running the exercises, or `list-users` may come back empty. Every command below runs **inside** the container.

Step 2 — Inspect the IAM environment

Task: Identify the existing users, groups and roles.

```
awslocal iam list-users --query 'Users[].UserName'
awslocal iam list-groups --query 'Groups[].GroupName'
awslocal iam list-roles --query 'Roles[].RoleName'
```

Expected result: Users `dev1`, `operator1`, `auditor1`; groups `Developers`, `Operators`, `ReadOnly`; role `AppRole`.

Step 3 — Create a student identity

Task: Create `student1` and `LabDevelopers`, then assign membership.

```
awslocal iam create-user --user-name student1
awslocal iam create-group --group-name LabDeve lopers
awslocal iam add-user-to-group --user-name student1 --group-name LabDeve lopers
awslocal iam list-groups-for-user --user-name student1
```

Expected result: LabDeve lopers is shown for student1.

Step 4 — Create a least-privilege policy

Task: Allow only s3:ListBucket and s3:GetObject.

```
cat >/tmp/student-s3-policy.json <<'EOF'
{"Version":"2012-10-17","Statement":[
{"Effect":"Allow","Action":["s3:ListBucket","s3:GetObject"],"Resource":["arn:aws:s3:::lab1-secure-data"],
{"Effect":"Allow","Action":["s3:ListBucket","s3:GetObject"],"Resource":["arn:aws:s3:::lab1-secure-data/*"]}]}
EOF
awslocal iam create-policy --policy-name StudentS3ReadOnly --policy-document file:///tmp/
student-s3-policy.json
```

Expected result: StudentS3ReadOnly appears under local policies.

Step 5 — Attach the policy

Task: Attach the read-only policy to LabDeve lopers.

```
POLICY_ARN=$(awslocal iam list-policies --scope Local --query "Policies[
?PolicyName=='StudentS3ReadOnly'].Arn | [0]" --output text)
awslocal iam attach-group-policy --group-name LabDeve lopers --policy-arn "$POLICY_ARN"
awslocal iam list-attached-group-policies --group-name LabDeve lopers
```

Expected result: StudentS3ReadOnly is attached.

Step 6 — Explicit DENY challenge

Task: Inspect the existing Developers policy and locate the DeleteObject DENY.

```
awslocal iam get-policy-version --policy-arn $(awslocal iam list-policies --scope Local --query
"Policies[?PolicyName=='DeveloperS3ReadOnly'].Arn | [0]" --output text) --version-id v1
```

Expected result: You identify the ExplicitDenyDelete statement and can explain why an explicit DENY always wins over any ALLOW.

Verification

Check	Expected
Step 2 — Inspect the IAM environment	Users dev1, operator1, auditor1; groups Deve lopers, Operators, ReadOnly; role AppRole
Step 3 — Create a student identity	LabDeve lopers is shown for student1
Step 4 — Create a least-privilege policy	StudentS3ReadOnly appears under local policies
Step 5 — Attach the policy	StudentS3ReadOnly is attached
Step 6 — Explicit DENY challenge	You identify the ExplicitDenyDelete statement and can explain why an explicit DENY always wins over any ALLOW

Completion check

- [] student1 and LabDeve lopers created
- [] Least-privilege S3 policy attached
- [] You can explain user vs group vs role
- [] You can explain explicit DENY

Cleanup

```
docker compose down -v
```

-v also removes the lab's volumes, so the next run starts from a clean state.

Reference

- Lab source code: <https://github.com/mohanpothula/aws-saa/tree/main/labs/lab-01-iam-account-security>
- Original student guide: [courseware/AWS_SAA_Lab1_Student_Guide.docx](#)
- [AWS IAM User Guide](#)
- [IAM security best practices \(least privilege\)](#)
- [IAM policy evaluation logic \(explicit deny\)](#)
- [IAM roles](#)
- [Amazon S3 User Guide](#)
- [LocalStack documentation](#)
- [LocalStack initialization hooks](#)
- [awslocal \(AWS CLI wrapper for LocalStack\)](#)